```
> file sealed-secrets.pem
sealed-secrets.pem: PEM certificate
```

Now let's create the same Sealed Secret using the above *sealed-secrets.pem:*

```
▫ kubeseal --cert ./sealed-secrets.pem -o yaml < secret.yaml
apiVersion: bitnami.com/v1alpha1
kind: SealedSecret
metadata:
  creationTimestamp: null
  name: mysecret
  namespace: addon-system
spec:
  encryptedData:
    password: AgCsa+ihNMcSpMrPrWnRO0VZQ6XZ0bl0crgOqQDTsimODe
BW
t5hHiGk0utl0zDyvOCofziWBf3uQpoMsB+MZCquBL7sF7FQiNLFK5EbZfWV3
Gr
kT3DN9YNyjDYunvy7TM9qlgJGLtHhQt3gTRwRozz2nz6UfJ92LI4mRfdm2eHTa
eTQy230pZJPp1Kw7bQWbrEq6rOkHbzQf3vsORlLUnmQO9DeJJM4cHddG7szIs
qUEeDJ6+uOMErmjOTvgPIoy1VVFnjcs9U5zdIVzsh6H28TErmcm+3krn+8DP/
ae0KpT+1WpSpmTsNU4EcXwYQHOZFP1eP0YBYUJ5yreg7Fu/LAOqlPs6GUWYc0
hN9bRZsPwcC5TfahO1nW8hvrtBjY9Zw0iTDNZQGcmcFE2BAQaQ/ri8VW9U8t+
yXWx7BQlmM071majTMBH9/d245FxUj4jtJ4NVE+wnk3fYxzNF0zChxni72EXW
Uww3PeI/QfSUfpqceMGnpbRud+qa1ycp24n6C6+pnqYQX+xGGBU6oLx5IK6l4
0eOnQbZzx2Qv6kH4Q8zLLqiIuF86FvXYP2St3FdzBZWecHK2uJuyR83gRDVn9
4Ksxv6wbALv5T6o+8AeQiSfdcE2gBVZbv20tmr0IrMzs961GSyT7LFLd+ujo
HyORj+c1yUYAUzsRksYI4wFokcwoweRPC+ZP0cK+XRYyzX5CErgl/s6KNYQ==
    username: AgCrtM/W5hJd8Ihg5GCh0lKnUK/
rhqU/4fNzKCVcnF98w1C+
VIy9M33sx2xGe8U4wqeYs1aPgPeqWznudHMGoLIuMYCt4X89dAF06o7w/
Hw0QHu /CbFFo3gToMj/V8uPRirhg09bCCyXQfmamVLYkGms
JWMLHs3uf9LATxpXZP35j EZvtaTd9uk4+oLVzc886R5H8ltFdwj
myPHCM76gXUSobomIMgds7EbHL6zYdbAOS1GOn25XZ4uCqIA ThtAcKM5n
jcm9dNlgL2LNaabPrR9wqug8vm+jjoJV59sBJaghKLyuuUH4vyaiqnfWh
sFnCnnucTwGlh/bQPU5xg+MSJgeE/aSvBhF/7syv1d+JXuQbdW1iQDjKE/
rjsVC5OHTd/x9/RyghyT0xDlUKTiy/xkzrtfQN3UOBwVGTqSPB5OUw1Rw
0WVXhM60tB0I+7OUxA1xtusYCwJKiJs7S7r0tMQ7MalS1o4/oiEm1fO/
rdxGiYD1KyaELLVDbudWKO9KXZ2lBgyplXqpdO3IuW0owGO
dWkBpn0XhQZ2SIKjec1boJ4GSIvEl8xQ5XExfzBenYyXoexQETsyyII38zLJ
kFACygJrh4SO1W6wixXWm9MAvDbySPxvKUHWgIuvHr5jTd+1N/kPy2Noz/
GFH7WYpIrPgoZfpFH/fJgjgvJEAzTad+PI+S74JA1VyQICO08ssCEkrS75
  template:
    data: null
    metadata:
      creationTimestamp: null
      name: mysecret
      namespace: addon-system
    type: Opaque
```

The above workflow will allow any of your teammates to create new sealed secrets without interacting with the cluster credentials. In fact, you can store the certificate publicly in a repository to allow others to create sealed secrets.

How it works

The underlying principle of Sealed Secrets is the usage of public key cryptography. The public certificate is used for sealing secrets. The private key the controller has is used for decrypting the sealed secrets.

Note that there are other implementation details that govern how sealed secrets work:

- The name space is used during the encryption process by default. Thus, two same secrets on different name spaces will have a different set of encrypted data within it.
- The secret name is also used during the encryption by default. This design decision disallows Sealed Secrets resources to be renamed. This improves the security as RBAC (role based access control) can enforce limitation of secret access by name and Sealed Secrets follows it by disallowing the secret to be renamed.

The above default behaviour can be configured. There is a concept of 'scopes' which has these rules:

- *strict* scope requires both name space and secret name as part of the encryption process.
- *namespace-wide* scope requires only a secret name as part of the encryption process.
- *cluster-wide* doesn't require either.

You could pass the —*scope* command line argument to the controller if you want to customise it.

By default, sealing keys are automatically renewed every 30 days. Note that this doesn't mean you have to re-encrypt your secrets every 30 days. All it means is that the controller will generate a new set of key pairs after 30 days. Any new sealed secret should be generated via the new certificate. But the old sealed secrets will continue to function since the old keys are not deleted — they are kept around and the new keys are just appended to them. END

Reference

GitHub repository: *https://github.com/bitnami-labs/sealed-secrets*

By: Sibi Prabakaran

The author is a senior engineer working at FPComplete.